Exercise Sheet 13

Before the Throne

House rules.

- First, inside the lesson folder: `pip install -r requirements.txt` (pandas, openpyxl, matplotlib).
- One file per exercise in `my_work/`. Everything you save goes to `my_work/output/`; create that folder at the top of every script with `Path("my_work/output").mkdir(parents=True, exist_ok=True)`.
- Every chart script starts with `import matplotlib` and `matplotlib.use("Agg")` **before** `import matplotlib.pyplot as plt`. Save with `plt.savefig(...)`; never `plt.show()`.
- Compare the printed output with the sheet, then **open the PNG or the workbook** and look at it. The file is the real result.
- Hand in: commit and push to `lesson-13`, open a pull request, and attach your favourite chart to it. The Privy Council likes pictures.

Exercise 1 Your first chart

10 min · `ex1_first_chart.py`

> **RAVEN #1301 from Queen Aldara, at dusk:** *"The Privy Council cannot read tables. We desire a picture of what the guild spends its days fighting. A bar thing. Blue, if possible. A."*

GIVEN

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import pandas as pd
from pathlib import Path

Path("my_work/output").mkdir(parents=True, exist_ok=True)
quests = pd.read_csv("data/quests.csv")
```

(a) Count quests per monster with `value_counts()`, biggest first, and print the result.

(b) Draw a bar chart of it (`plt.bar(counts.index, counts.values)`), with a title and a y-axis label. Rotate the monster names 30° so they don't overlap: `plt.xticks(rotation=30, ha="right")`.

(c) Call `plt.tight_layout()`, then save as `my_work/output/quests_by_monster.png` at `dpi=150`. Open it.

EXPECTED OUTPUT

```
monster
Goblins          81
Bandits          72
Wolves           64
Ghouls           61
Giant Rats       42
Giant Spiders    42
Trolls           31
Wyrm              7
Name: count, dtype: int64
Saved my_work/output/quests_by_monster.png
```

Hint: The Queen's blue is `color="#1f3a5f"`. Any colour name works too: `"steelblue"`, `"orange"`.

Exercise 2 What does a quest pay?

10 min · `ex2_reward_histogram.py`

> **RAVEN #1302 from Captain Siv:** *"Before I post another notice I want to see the rewards, not scroll through four hundred of them. How many quests pay roughly what? Where's the middle? Yes or no will do. (It won't, I know.)"*

Same Given block as Exercise 1.

(a) Print the smallest, biggest and median `reward`, formatted as crowns (`cr`) with thousands separators.

(b) Draw a histogram of `reward` with `bins=20`. Add a dashed vertical line at the median: `plt.axvline(median, linestyle="--", label="Median")` and `plt.legend()`.

(c) Label both axes, add a title, save as `my_work/output/rewards.png`.

EXPECTED OUTPUT

```
Smallest reward: 90 cr
Biggest reward:  4,540 cr
Median reward:   470 cr
Saved my_work/output/rewards.png
```

Hint: A histogram is a bar chart of *how many* values fall into each bucket. Excel can do one, but only since 2016 and only if you find the button. Look at the lonely bars far to the right: who posts quests like that?

Exercise 3 Side by side

15 min · `ex3_side_by_side.py`

> **RAVEN #1303 from Quartermistress Pell:** *"How do quests end, per difficulty? Success, fled, failed, next to each other, one picture. The Privy Council sits in a fortnight and I'd like them to notice something. P."*

Same Given block, plus `import numpy as np`.

(a) Build a pivot table: difficulty as rows, `outcome` as columns, counting `quest_id` (`aggfunc="count"`). Print it.

(b) Grouped bars: `x = np.arange(len(table.index))`, `width = 0.27`. Draw Success at `x - width`, Fled at `x` and Failed at `x + width`, each with a `label=`. Put the difficulties on the ticks with `plt.xticks(x, table.index)`.

(c) Axis labels, title, legend, `figsize=(9, 4.5)`, save as `my_work/output/outcomes_by_difficulty.png`.

(d) Look at the picture. At which difficulty do the dark bars stop winning comfortably?

EXPECTED OUTPUT

```
outcome     Failed  Fled  Success
difficulty
1                1     3       30
2               11    17       82
3               24    24       80
4               11    27       50
5               12    11       17
Saved my_work/output/outcomes_by_difficulty.png
```

Exercise 4 Back to Excel

10 min · `ex4_first_workbook.py`

> **RAVEN #1304 from Orrin of the Clockworks:** *"The Queen's chamberlain asked if the Python thing 'does Excel'. I said yes. Please make me right."*

GIVEN

```
import pandas as pd
from pathlib import Path

Path("my_work/output").mkdir(parents=True, exist_ok=True)
quests = pd.read_csv("data/quests.csv")
quests["is_success"] = quests["outcome"] == "Success"
```

(a) Group by `monster` and build a table with `quests` (count of `quest_id`), `successes` (sum of `is_success`) and `avg_reward` (mean of `reward`), rounded to 0. Sort by quests, biggest first.

(b) Add a column `success_rate` = successes / quests, rounded to 3 decimals. Print the table.

(c) Save it with `to_excel("my_work/output/monsters.xlsx", sheet_name="Monsters")`. Open the workbook.

EXPECTED OUTPUT

```
               quests  successes  avg_reward  success_rate
monster
Goblins            81         55       313.0         0.679
Bandits            72         50       553.0         0.694
Wolves             64         45       349.0         0.703
Ghouls             61         33       722.0         0.541
Giant Spiders      42         29       834.0         0.690
Giant Rats         42         34       145.0         0.810
Trolls             31         13      1309.0         0.419
Wyrm                7          0      3130.0         0.000
Saved my_work/output/monsters.xlsx
```

Hint: `True` counts as 1 when you sum it. That's why `is_success` is handy. And yes: seven Wyrm quests, not one success. Hold that thought until next lesson.

Exercise 5 One workbook, several sheets

10 min · `ex5_several_sheets.py`

> **RAVEN #1305 from Quartermistress Pell:** *"One file, please, not six. Tabs are fine. Tabs are great. Dalvric once sent me fourteen workbooks called final.xlsx. By raven. Separately."*

Same Given block as Exercise 4 (without `is_success`), plus `heroes.csv` and `party.csv`.

(a) Make three tables: reward `count`, `mean`, `median` by monster; the same by region (both rounded to 0); and the top 10 heroes by loot: merge `party` with `heroes` on `hero_id`, sum `loot_share` per `hero_id, name, class`, round to 2, `reset_index()`, biggest first.

(b) Write all three into `my_work/output/guild_report.xlsx` using `with pd.ExcelWriter(path) as writer:` and one `to_excel(writer, sheet_name=...)` per table. The top 10 sheet gets `index=False`.

(c) Print the region table and a line naming the sheets.

EXPECTED OUTPUT

```
               count   mean  median
region
Greyfen           68  502.0   450.0
Ironhills         85  662.0   550.0
Northern Wilds    75  732.0   610.0
Saltmarsh         82  480.0   390.0
```

```
Thornwood          90  561.0   405.0
Sheets written: By monster, By region, Top 10 heroes
```

Exercise 6 Make it pretty

15 min · `ex6_make_it_pretty.py`

> **RAVEN #1306 from Queen Aldara:** *"Lovely numbers. Could the top row be bold? And blue? And could the crowns look like crowns? And could the columns not be squashed? We thank you ever so. A."*

GIVEN

```
import pandas as pd
from openpyxl import load_workbook
from openpyxl.styles import Font, PatternFill
from pathlib import Path

Path("my_work/output").mkdir(parents=True, exist_ok=True)
quests = pd.read_csv("data/quests.csv")
```

(a) Per monster: `quests` (count), `avg_reward` (mean), `total_paid` (sum of `paid`), sorted by average reward, highest first. Add `paid_per_quest` = total paid / quests. Save to `my_work/output/pretty_report.xlsx`, sheet `Monsters`.

(b) Re-open it with `load_workbook(path)`, pick the sheet with `wb["Monsters"]`, and for every cell in row 1 set `Font(bold=True, color="FFFFFF")` and `PatternFill("solid", fgColor="1F3A5F")`.

(c) Give columns C to E the number format `'#,##0.00 "cr"'` (single quotes outside, because the format itself contains double quotes): loop over `ws.iter_rows(min_row=2, min_col=3, max_col=5)`.

(d) Set column A to width 16 and B–E to 15 (`ws.column_dimensions["A"].width`), freeze the header with `ws.freeze_panes = "A2"`, save, and print the checks below.

EXPECTED OUTPUT

```
Rows: 9, columns: 5
A1 bold: True, C2 format: #,##0.00 "cr"
Top of the table: Wyrm, 3,130.00 cr on average
Saved my_work/output/pretty_report.xlsx
```

Hint: `ws.max_row`, `ws.max_column`, `ws["A1"].font.bold`, `ws["C2"].number_format` and `ws["A2"].value` give you what to print. Then open the workbook and read the Wyrm's `paid_per_quest`.

Exercise 7 A chart inside the workbook STRETCH

15 min · `ex7_chart_in_excel.py`

> **RAVEN #1307 from Queen Aldara:** *"Can the picture live in the Excel? The Privy Council likes it when things are in the Excel. A."*

(a) Average reward per region, rounded, sorted smallest to largest. Print it. Draw a **horizontal** bar chart (`plt.barh`) and save it as `my_work/output/avg_reward.png`.

(b) Write the table to `my_work/output/report_with_chart.xlsx`, sheet `Data` (`avg.to_frame("avg_reward").to_excel(...)`).

(c) Re-open the workbook, `ws = wb.create_sheet("Chart")`, then `ws.add_image(Image("my_work/output/avg_reward.png"), "B2")` with `from openpyxl.drawing.image import Image`. Save. Print `wb.sheetnames`.

EXPECTED OUTPUT

```
region
Saltmarsh         480.0
Greyfen           502.0
Thornwood         561.0
```

```
Ironhills         662.0
Northern Wilds    732.0
Name: reward, dtype: float64
Sheets: ['Data', 'Chart']
Saved my_work/output/report_with_chart.xlsx
```

Bug hunt Pages from the Grimoire, page 13

15 min · `bughunt/report_bug.py`

Dalvric's last working before he sailed west: "the Privy Council's pack in one click". The Queen clicked. Nothing happened. There are **three** bugs: one crashes, one shows the wrong heroes with a straight face, and one gives a number that's a bit too low (think about failed quests).

EXPECTED OUTPUT ONCE FIXED

```
Top 3 heroes by loot:
H008 Jasper Ironfoot 4,428.68 cr
H070 Benedek Saltmarsh 3,833.34 cr
H019 Silas Goldenhand 3,502.33 cr
Average share when a quest paid: 145.86 cr
Saved my_work/output/council_pack.xlsx
```

One commit per bug, with a message that explains it.

More riddles. Five more torn pages wait in `bughunt/riddles/`, one bug each (the last one has two), getting harder as you go. Optional: for fast finishers, or a quiet evening.